

Causal ordering

Contents

1. Problem definition
2. Why causal order is important?
3. Causal ordering protocols based on vector clock
 - 3.1. Birman-Schiper-Stephenson
 - 3.2. Schiper-Eggli-Sandoz

System model

- Distributed system consisting of a collection of N processes P_i , $i=1, 2, \dots, N$
- Each process executes on a single processor
- Processes communicate only by message passing
- Each process P_i has a state s_i
- Each process executes a sequence of events
- There are three types of events: send, receive and operation that transforms the state
- There is no global time

Problem definition

- Often in a distributed system it is necessary to be able to answer the question:
 - has event e_1 at process P_1 been responsible for causing event e_2 at process P_2 ?
- Having the answer one can establish the relationships between events, i.e. the order in which the events take place
- Causal order denotes a partial ordering defined over a set of events
- Causal order is based on happened before relation

Logical clocks

- Scalar clocks

- $a \rightarrow b$ implies $C(a) < C(b)$
- $C(a) < C(b)$ not necessarily implies $a \rightarrow b$, i.e. simply looking at the event clocks one cannot decide if the events may be or may not be causally related

- Vector clocks

- $a \rightarrow b$ implies $VC(a) < VC(b)$
- $VC(a) < VC(b)$ implies $a \rightarrow b$, i.e. simply looking at the event clocks one can decide if the events may be or may not be causally related

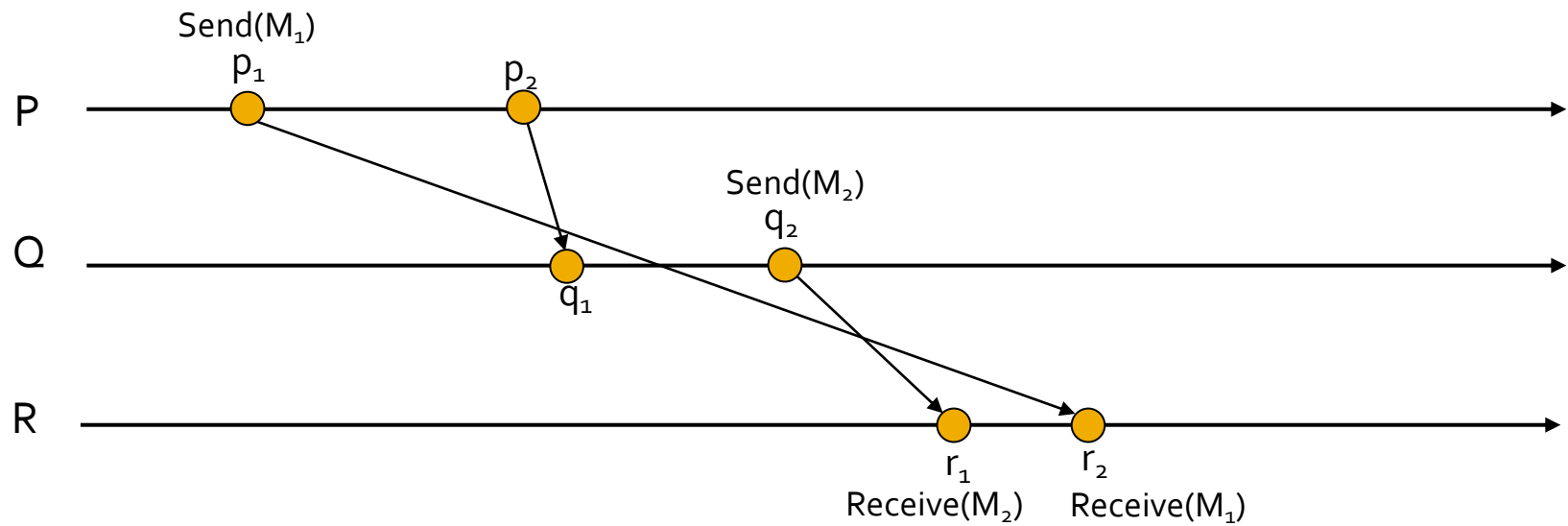
Causal ordering of events and messages

- So far we have discussed about causal ordering of events, i.e. specifying which of two events occurred first
- Causal ordering of messages implies that the causal relationship among "message send" events correspond to "message receive" events

Causal ordering of events and messages

- Message delivery is said to be causal if the order in which messages are received is consistent with the order in which they are sent.
- If $\text{Send}(M_1) \rightarrow \text{Send}(M_2)$, then $\text{Receive}(M_1) \rightarrow \text{Receive}(M_2)$ if M_1 and M_2 are received by the same process, i.e. all messages are processed in the order that they were created
- A message is dependent on other messages that were sent before it
- The message can be delivered only when no causality constraints are violated, otherwise, the message is not delivered immediately but is buffered until all the preceding messages are delivered

Violation of causal ordering of messages



Why causal order is important?

- Maintaining consistency in replicated databases
- Monitoring a distributed system
- Resource allocation

Causal ordering protocols

- Causal ordering protocol based on causal history
- Causal ordering protocols based on clocks
 - physical clock (common clock or perfectly synchronized clock)
 - vector clocks

Causal ordering protocol based on vector clocks

Idea: A process receives a message sent, delays it if is necessary, and then delivers it in an order according to causality. Therefore a message is delivered only if the messages preceding it have been delivered. Otherwise the message is buffered.

Algorithms:

Birman-Schiper-Stephenson

Schiper-Eggli-Sandoz

Birman-Schiper-Stephenson algorithm

- The set of processes communicate using broadcast
- $P_1, P_2, \dots, P_n$ processes
- t^m is the vectorstamp for message m
- each process P_i has:
 - a (kind of) vector clock VC_i

Birman-Schiper-Stephenson algorithm

- C is not exactly a vector clock as the updating procedure does not entirely follow the one for the vector clocks.
- There are three basic principles to this algorithm:
 - All messages are time stamped by the sending process.
[Note: This time is separate from the global time talked about in the previous section. Instead, each element of the vector corresponds to the number of messages sent (including this one) to other processes.]
 - A message can not be delivered until:
 - All the messages before this one have been delivered locally.
 - All the other messages that have been sent out from the original process has been accounted as delivered at the receiving process.
 - When a message is delivered, the clock is updated.

Birman-Schiper-Stephenson algorithm

P_i broadcasts a message m

- P_i increments $VC_i[i]$ and sets the timestamp $t^m = VC_i$
- $VC_i[i] - 1$ indicates how many messages from P_i precede m

Birman-Schiper-Stephenson algorithm

P_j ($j \neq i$) receives a message m with timestamp t^m from P_i ; it delays the delivery of the message until:

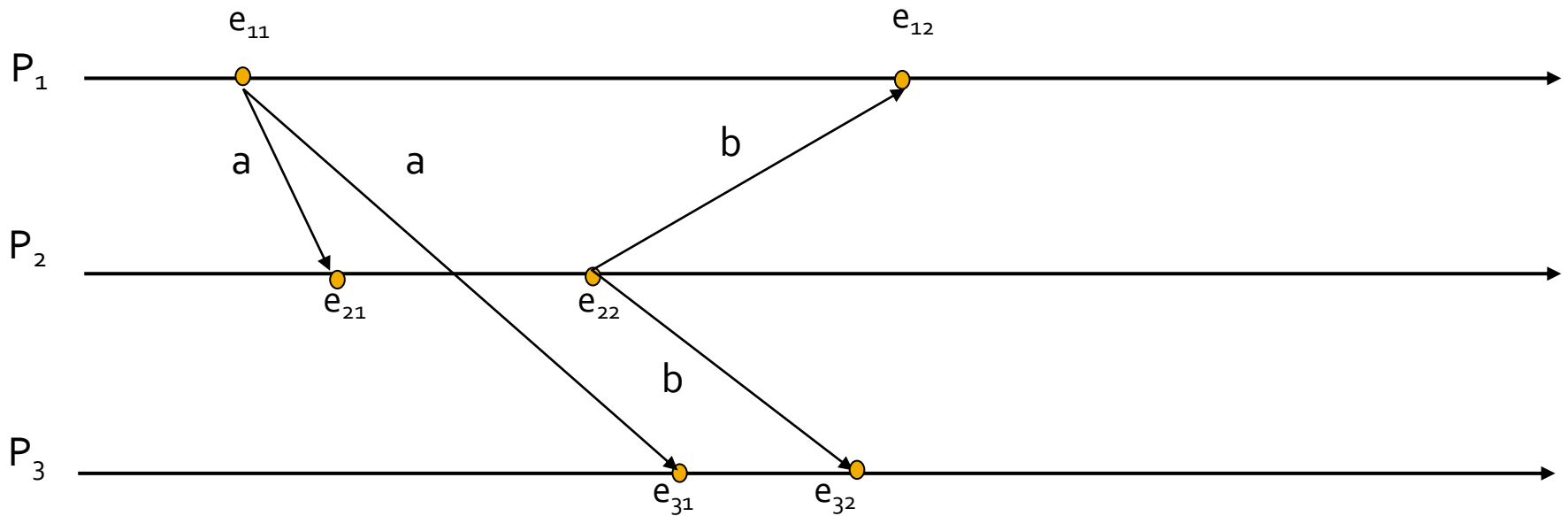
1. $VC_j[i] = t^m[i] - 1$ (this step checks that P_j has received all the messages from P_i that precede m)
2. for all $k \leq n$, $k \neq i$, $VC_j[k] \geq t^m[k]$ (this step checks that P_j has received all those messages received by P_i before sending m)

Birman-Schiper-Stephenson algorithm

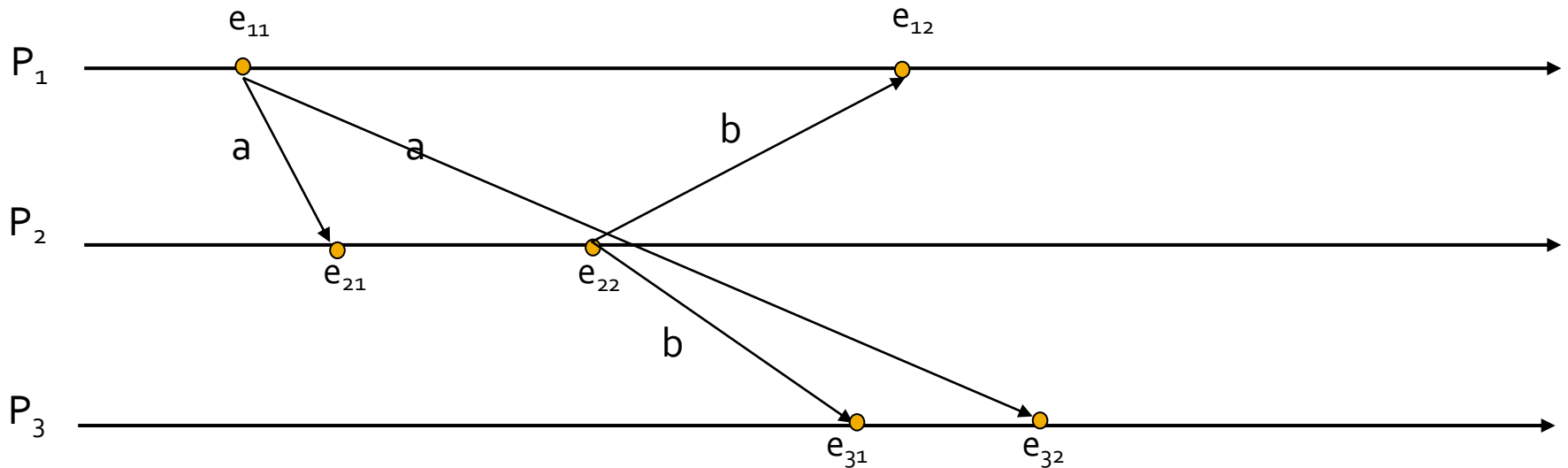
When the message is delivered to P_j

- P_j updates its vector clock as follows:
$$VC_j[k] = \max(VC_j[k], t^m[k])$$
- checks buffered messages to see if any can be delivered

Birman-Schiper-Stephenson algorithm



Birman-Schiper-Stephenson algorithm



Schipper-Eggli-Sandoz algorithm

- does not require broadcast
- $P_1, P_2, \dots, P_n$ processes
- each process P_i has
 - a vector clock VC_i
 - a vector of tuple V_i having length of $n-1$
 - tuple (j, t^m) where j is the destination process and t^m the vector timestamp of the last message sent by P_i to P_j

Schipper-Eggli-Sandoz algorithm

- VC is a vector clock.
- Instead of maintaining a vector clock based on the number of messages sent to each process, the vector clock for this protocol can increment at any rate it would like to and has no additional meaning related to the number of messages currently outstanding.
- **Sending a message:**
 - All messages are timestamped and sent out with a list of all the timestamps of messages sent to other processes.
 - Locally store the timestamp that the message was sent with.

Schiper-Eggli-Sandoz algorithm

■ Receiving a message:

- A message cannot be delivered if there is a message mentioned in the list of timestamps that precedes this one.
- Otherwise, a message can be delivered, performing the following steps:
 - 1. Merge in the list of timestamps from the message:
 - - Add knowledge of messages destined for other processes to our list of processes if we didn't know about any other messages destined for one already.
 - - If the new list has a timestamp greater than one we already had stored, update our timestamp to match.
 - 2. Update the local logical clock.
 - 3. Check all the local buffered messages to see if they can now be delivered.

Schiper-Eggli-Sandoz algorithm

V_i

$$\left(\begin{array}{ll} 1: & t_{p_1}^m \\ 2: & t_{p_2}^m \\ \dots & \dots \\ i-1: & t_{p_{i-1}}^m \\ i+1: & t_{p_{i+1}}^m \\ \dots & \dots \\ n: & t_{p_n}^m \end{array} \right)$$

destination process

vector timestamp of the last message sent by

P_i to $P_1, P_2, \dots, P_{i-1}, P_{i+1}, \dots, P_n$

Schiper-Eggli-Sandoz algorithm

P_i send a message to P_j

1. P_i increments $VC_i[i]$ and set $t_{Pj}^m = VC_i$
2. P_i sends m timestamped t^m and V_i to P_j
3. $V_i[j] = t_{Pj}^m$

Pair (P_j, t_{Pj}^m) was not sent to P_j

Schiper-Eggli-Sandoz algorithm

P_j ($j \neq i$) receives a message m with timestamp t^m and V_i from P_i :

$V_i[j]$ is not set $\rightarrow$ message can be delivered

$V_i[j]$ is set and $V_i[j] \leq VC_j \rightarrow$ message can be delivered

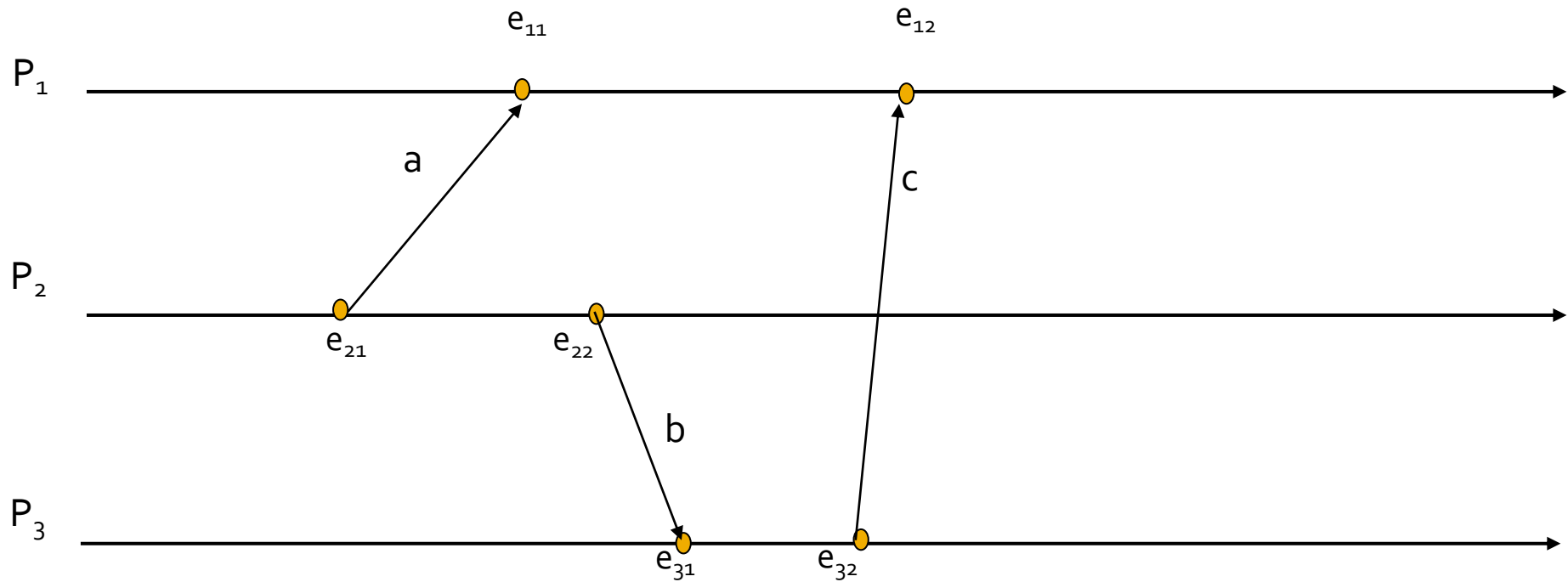
$V_i[j]$ is set and $V_i[j] > VC_j \rightarrow$ message cannot be delivered (some events occurred at other processes, but P_j does not know about)

Schipper-Eggli-Sandoz algorithm

When the message is delivered to P_j

- P_j updates the elements of V_j with the corresponding elements of V_i , except for $V_j[j]$, as follows
 - if $V_j[k]$ is not initialized then $V_j[k] = V_i[k]$
 - if $V_j[k]$ is initialized then $V_j[k] = \max(V_j[k], V_i[k])$
- P_j updates its vector clock (using $t_{P_j}^m$)
- checks buffered messages to see if any can be delivered

Schipper-Eggli-Sandoz algorithm



Schipper-Eggli-Sandoz algorithm (Homework)

